Reporte: Comunicación P2P por Internet sin Intermediario — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar cómo los dispositivos pueden comunicarse por internet sin un servidor central, qué opciones existen, seguridad, y viabilidad para Crozzo POS.

---

1. El problema: ¿se puede comunicar sin intermediario?

Respuesta corta: NO completamente. Pero sí se puede minimizar el intermediario.

Toda comunicación P2P por internet requiere **al menos un mecanismo de descubrimiento inicial** (signaling). La diferencia está en **quién controla** ese mecanismo:

Modelo	Intermediario	Control	Costo
Servidor central (Supabase)	Supabase	Empresa	$$
WebRTC + signaling propio	Tu servidor	Tú	$
WebRTC + Nostr relays	Relays públicos	Nadie (descentralizado)	Gratis
WebRTC + DHT (BitTorrent)	Ninguno	Nadie	Gratis
GitHub como relay	GitHub	GitHub	Gratis (limitado)

---

2. Opciones analizadas

2.1 WebRTC (el estándar P2P del navegador)

Qué es: Protocolo que permite comunicación directa entre peers (navegadores/dispositivos) después de un handshake inicial.

Cómo funciona:
```
1. Peer A genera "offer" (SDP + ICE candidates)
2. Peer A envía offer a Peer B (vía signaling)
3. Peer B genera "answer" y lo envía a Peer A (vía signaling)
4. Ambos intercambian ICE candidates (rutas de red)
5. Conexión P2P establecida → datos fluyen directamente
6. Signaling ya no se necesita
```

Ventaja: Después del handshake, los datos van directamente entre peers. No pasa por servidor. Latencia mínima.

Desventaja: Necesita signaling inicial. Sin signaling, los peers no se encuentran.

Viabilidad en Crozzo: ✅ WebRTC está disponible en el webview de Tauri (Chromium/WebView2). Se puede usar `RTCPeerConnection` desde JS.

Seguridad: DTLS-SRTP (cifrado end-to-end). Los datos no pasan por servidor después del handshake.

2.2 Signaling sin servidor central

El handshake inicial (signaling) es el cuello de botella. Opciones para hacerlo sin servidor propio:

A. QR Code (ya implementado en Crozzo)
```
```

```
Peer A genera QR con su offer (SDP + ICE)
Peer B escanea QR → obtiene offer → genera answer
Peer B muestra QR con answer
Peer A escanea → conexión establecida
```
- **Ventaja**: Cero infraestructura. Funciona offline (misma red).
- **Desventaja**: Requiere que ambos estén físicamente cerca (para escanear QR). No funciona cross-network.
- **Estado en Crozzo**: ✅ Ya implementado (CrozzoPairingSeal.js)

B. Nostr Relays (descentralizado)
```
Peer A publica su offer como evento Nostr firmado
Relay público (wss://nos.lol) distribuye el evento
Peer B se suscribe a eventos del Peer A → recibe offer
Peer B publica answer → Peer A lo recibe
Conexión P2P establecida
```
- **Ventaja**: Sin servidor propio. Relays son públicos y gratuitos. Censura-resistente.
- **Desventaja**: Depende de relays públicos (pueden caer, rate-limit). Latencia mayor.
- **Seguridad**: Eventos firmados con clave privada (secp256k1). Nadie puede falsificar.
- **Viabilidad**: ✅ Se puede implementar con librería Nostr (nostr-tools). Los relays son WebSocket estándar.
- **Complejidad**: Media. Requiere integrar Nostr client + WebRTC.

C. DHT (BitTorrent-style, sin servidor)
```
Peer A publica su offer en la DHT (tabla hash distribuida)
Peer B busca en la DHT por clave → encuentra offer
Conexión P2P establecida
```
- **Ventaja**: Totalmente descentralizado. Sin servidor. Sin relays.
- **Desventaja**: Latencia alta (descubrimiento puede tardar segundos). Complejidad de implementación. No disponible en navegador nativo.
- **Viabilidad en Tauri**: ⚠️ Requiere implementar DHT en Rust (libp2p). Complejo.
- **Recomendación**: ❌ No recomendado para POS. Overkill.

D. GitHub como relay (el ejemplo del usuario)
```
Peer A crea un "gist" o "issue" con su offer (JSON firmado)
Peer B lee el gist → obtiene offer → crea answer
Peer B crea gist con answer
Peer A lee → conexión establecida
```
- **Ventaja**: Gratis. GitHub tiene alta disponibilidad. Fácil de implementar (API REST).
- **Desventaja**: Latencia alta (polling). Rate limits (5000 req/hora autenticado). No es tiempo real. GitHub puede bloquear uso no previsto.
- **Seguridad**: Datos firmados con clave del dispositivo. GitHub solo almacena, no puede leer (si se cifra).
- **Viabilidad**: ✅ Fácil de implementar (fetch a GitHub API).
- **Recomendación**: ⚠️ Solo para casos específicos (backup/transferencia, no sync en tiempo real).

---

3. Comparativa de opciones

Opción	Latencia	Infraestructura	Complejidad	Seguridad	Tiempo real
Supabase (actual)	Media	Servidor propio	Baja	Media (TLS)	✅
WebRTC + QR	Muy baja	Ninguna	Baja	Alta (DTLS)	✅ (misma red)
WebRTC + Nostr	Baja	Relays públicos	Media	Alta (firmado)	✅

WebRTC + DHT	Alta	Ninguna	Muy alta	Alta	⚠️
GitHub relay	Alta	GitHub	Baja	Media (cifrado)	❌
WebRTC + Supabase signaling	Baja	Supabase	Media	Alta	✅

---

4. Seguridad en P2P

4.1 Capas de seguridad

```
CAPA 1 — Identidad
  └→ Clave pública/privada por dispositivo (ECDH P-256 o secp256k1)
  └→ Cada mensaje firmado con clave privada
  └→ Receptor verifica firma con clave pública

CAPA 2 — Cifrado de transporte
  └→ WebRTC: DTLS-SRTP (cifrado end-to-end)
  └→ Sin WebRTC: cifrar payload con AES-256-GCM antes de enviar

CAPA 3 — Autenticación
  └→ Token de pareo (QR) como secreto compartido
  └→ Validación de identidad del peer antes de aceptar conexión

CAPA 4 — Autorización
  └→ Roles (caja, mesero, admin) determinan qué datos se comparten
  └→ Un mesero no recibe datos de configuración de admin

CAPA 5 — Integridad
  └→ Hash de cada mensaje (SHA-256)
  └→ Verificación de que no fue alterado en tránsito
```

4.2 Riesgos y mitigaciones

Riesgo	Mitigación
Man-in-the-middle en signaling	Firmar offer/answer con clave del dispositivo
Relay malicioso (Nostr/GitHub)	Cifrar payload antes de enviar; relay solo transporta
Peer falso se conecta	Validar token de pareo + firma
Datos interceptados en tránsito	DTLS-SRTP (WebRTC) o AES-256-GCM (manual)
Replay attack	Timestamp + nonce en cada mensaje
Denial of service	Rate limiting + validación de origen

---

5. Estrategia recomendada para Crozzo POS

Fase 1 — Mejorar lo que ya existe (1-2 semanas)
```
1. Corregir bugs latentes (digest, WS panic, body truncation)
2. Agregar WebRTC como capa adicional (signaling vía Supabase)
3. Data channel para sync de datos en tiempo real
4. Fallback automático: WebRTC → HTTP/WS → Cloud → Offline
```

Fase 2 — Signaling descentralizado (2-3 semanas)
```
1. Integrar Nostr relays como signaling (sin servidor propio)
2. Publicar offer/answer como eventos Nostr firmados
3. Peers se encuentran por relays públicos
```

4. Conexión P2P directa establecida
5. Datos fluyen sin pasar por Supabase
```

### Fase 3 — Transferencia por link (1 semana)
```
1. Endpoint para generar link de descarga (estilo wormhole)
2. Compartir link → otro dispositivo descarga
3. Útil para reportes/backups sin saturar DB
4. Funciona cross-network
```

### Prioridad de conexión (automática)
```
1. ¿Misma red LAN? → mDNS + HTTP/WS (más rápido, ya implementado)
2. ¿Hay internet + WebRTC disponible? → WebRTC P2P (baja latencia)
3. ¿Hay internet sin WebRTC? → Cloud sync (Supabase)
4. ¿Sin red? → Offline (datos locales)
```

---

## 6. Lo que NO es realista

| Idea | Por qué no |
|---|---|
| "Sin ningún intermediario" | Siempre se necesita signaling inicial (QR, DHT, relay) |
| "GitHub como server de sync" | Rate limits, latencia, no tiempo real, TOS |
| "Bluetooth mesh para datos" | Ancho de banda muy bajo, alcance limitado |
| "WiFi Direct automático" | No soportado en web/Tauri, complejidad OS |
| "P2P sin signaling" | Imposible en internet (NAT traversal requiere ICE) |

---

## 7. Conclusión

**Sí es posible comunicarse por internet sin servidor central**, pero siempre se necesita un mecanismo de descubrimiento inicial. Las opciones viables son:

1. **WebRTC + Nostr relays**: La mejor opción. Signaling descentralizado, datos P2P directos, cifrado end-to-end. Costo: solo relays públicos (gratis).
2. **WebRTC + Supabase signaling**: Lo que ya tenemos parcialmente. Signaling vía Supabase, datos P2P directos.
3. **QR code**: Para emparejamiento inicial sin infraestructura. Ya implementado.

**Lo que NO se puede hacer:**
- Comunicación 100% sin signaling (imposible en internet)
- Usar GitHub como servidor de sync en tiempo real (rate limits)
- Bluetooth mesh para grandes volúmenes de datos

**Recomendación**: Implementar WebRTC con signaling vía Supabase (Fase 1), luego migrar a Nostr relays (Fase 2) para eliminar dependencia de servidor propio. Mantener cloud sync como fallback. Todo con cifrado end-to-end (DTLS-SRTP).
```